30-Day PostgreSQL for Data Engineering

Day 6 — String Functions

Session	Day 6 of 30
Topic	String Functions
Week	Week 1 — SQL Foundations
Duration	~35 minutes
Level	Complete Beginner
Goal	Clean and transform messy text data using PostgreSQL string functions

What You Will Learn Today

- LOWER, UPPER, INITCAP — normalize casing for consistent storage and JOINs
- TRIM, LTRIM, RTRIM — strip hidden whitespace that breaks comparisons
- CONCAT and || — join strings; understand how they differ with NULLs
- LENGTH and SUBSTRING — measure and slice strings for validation and extraction
- LIKE and ILIKE — pattern matching in WHERE clauses
- REPLACE and REGEXP_REPLACE — find-and-swap characters with regex power

Session Agenda

Time	Segment	Content
0:00 – 2:00	Hook	Why messy strings break pipelines
2:00 – 7:00	Concept 1	LOWER, UPPER, INITCAP — casing
7:00 – 12:00	Concept 2	TRIM, LTRIM, RTRIM — whitespace
12:00 – 17:00	Concept 3	CONCAT and || — joining strings
17:00 – 21:00	Concept 4	LENGTH and SUBSTRING — slicing
21:00 – 26:00	Concept 5	LIKE and ILIKE — pattern matching
26:00 – 30:00	Concept 6	REPLACE and REGEXP_REPLACE
30:00 – 35:00	Exercise	Clean a full raw_customers table

■ 0:00 – 2:00 | Hook — Why Strings Break Pipelines

Camera Script:

"Real data is messy. Names come in with random capitalization. Email addresses arrive in UPPERCASE from one system and lowercase from another. Phone numbers have parentheses, dashes, dots — sometimes all three in the same column. Today's lesson is about fixing all of that before it reaches your clean table. These are the string functions you'll use in literally every pipeline you ever build."

Key Message to Deliver

- Messy strings are the #1 data quality issue in real pipelines
- PostgreSQL has a rich set of string functions — you don't need Python for this
- These functions are applied in the SELECT (transform) layer of your ELT pipeline
- The exercise today is a real staging-layer transform, not a toy example

■ 2:00 – 7:00 | Concept 1 — LOWER, UPPER, INITCAP

These three functions change the casing of a string. They're simple but critical.

Function	Description	Example Input	Output
LOWER(str)	All lowercase	'Hello WORLD'	'hello world'
UPPER(str)	All uppercase	'Hello WORLD'	'HELLO WORLD'
INITCAP(str)	First letter of each word capitalized	'hello world'	'Hello World'

Demo Code:

```
SELECT
  LOWER('Hello WORLD')    AS lowercased,
  UPPER('Hello WORLD')    AS uppercased,
  INITCAP('hello world')  AS title_cased;
```

Pipeline Context:

```
-- Normalize emails on ingest (before storing)
SELECT
  email_raw,
  LOWER(email_raw) AS email_normalized
FROM (VALUES
  ('Alice@Gmail.COM'),
  ('BOB@YAHOO.com')
) AS t(email_raw);
```

Why LOWER on emails?

- alice@gmail.com and ALICE@Gmail.COM are the same person
- Without normalization, a JOIN on email produces 0 matches
- Always LOWER email addresses before storing in any table used for JOINs

■ 7:00 – 12:00 | Concept 2 — TRIM, LTRIM, RTRIM

Whitespace at the edges of a string is invisible but breaks everything: comparisons, JOINs, and storage. TRIM removes it.

Function	What it removes
TRIM(str)	Leading AND trailing spaces
LTRIM(str)	Leading (left-side) spaces only
RTRIM(str)	Trailing (right-side) spaces only
TRIM(char FROM str)	A specific character from both ends

```
SELECT
  TRIM('   hello   ')         AS trimmed,
  LTRIM('   hello   ')        AS left_trimmed,
  RTRIM('   hello   ')        AS right_trimmed,
  TRIM('$' FROM '$$$99.99$$$') AS custom_trim;
```

Pro Tip — Use LENGTH to reveal hidden whitespace

- LENGTH(' hello ') = 11 -- includes the spaces
- LENGTH(TRIM(' hello ')) = 5 -- the real content
- When debugging data issues, compare LENGTH() before and after TRIM()

■ 12:00 – 17:00 | Concept 3 — CONCAT and ||

Two ways to join strings together. Choose carefully when NULLs are involved.

Method	NULL behavior	Example
|| operator	NULL + anything = NULL	'a' || NULL = NULL
CONCAT()	NULLs are silently skipped	CONCAT('a', NULL, 'b') = 'ab'

```
SELECT
  'Hello' || ' ' || 'World'         AS pipe_operator,
  CONCAT('Hello', ' ', 'World')     AS concat_function,
  CONCAT('user_', 42)              AS mixed_types,
  'Hello' || NULL                   AS pipe_with_null,
  CONCAT('Hello', NULL, 'World')    AS concat_with_null;
```

Pipeline Use Case — Composite Keys:

```
-- Build a surrogate key for a staging table
SELECT
  region || '_' || product_id  AS composite_key
FROM (VALUES
  ('US', 'PROD001'),
  ('EU', 'PROD002')
) AS t(region, product_id);
```

■ 17:00 – 21:00 | Concept 4 — LENGTH and SUBSTRING

LENGTH counts characters. SUBSTRING slices a portion out. PostgreSQL string positions start at 1 (not 0).

```
SELECT
  LENGTH('pipeline')                    AS char_count,   -- 8
  SUBSTRING('pipeline' FROM 1 FOR 4)    AS first_four,   -- 'pipe'
  SUBSTRING('pipeline' FROM 5)          AS from_pos_5,   -- 'line'
  SUBSTRING('2024-01-15' FROM 1 FOR 4)  AS year_part;    -- '2024'
```

Pipeline Use Cases

- Validate that a product code is exactly 8 characters with LENGTH()
- Extract the year from a date stored as TEXT with SUBSTRING()
- Pull a 3-letter prefix from a product code to route it to the right table

■ 21:00 – 26:00 | Concept 5 — LIKE and ILIKE

Pattern matching inside WHERE. LIKE is case-sensitive; ILIKE (PostgreSQL-specific) is case-insensitive.

Wildcard	Meaning	Example
%	Any sequence of characters (including none)	'%@gmail.com' matches 'a@gmail.com'
_	Exactly one character	'SKU___' matches 'SKU001' but not 'SKU0001'

```
-- LIKE is case-sensitive
WHERE email LIKE '%@gmail.com'

-- ILIKE is case-insensitive (PostgreSQL-only)
WHERE name ILIKE 'alice%'   -- matches Alice, ALICE, alice123

-- Underscore: exactly one character
WHERE product_code LIKE 'SKU___'  -- SKU + 3 chars
```

■ 26:00 – 30:00 | Concept 6 — REPLACE and REGEXP_REPLACE

```
-- REPLACE: swap a fixed string
SELECT REPLACE('$1,234.56', ',', '');   -- '$1234.56'
SELECT REPLACE('$1,234.56', '$', '');   -- '1,234.56'

-- REGEXP_REPLACE: pattern-based
-- Syntax: REGEXP_REPLACE(string, pattern, replacement, flags)
-- 'g' flag = global (replace ALL matches)
SELECT REGEXP_REPLACE('(555) 123-4567', '[^0-9]', '', 'g');
-- Result: '5551234567'
```

Regex Cheat Sheet (Most Used in Pipelines)

- [^0-9] — anything that is NOT a digit (use with '' to strip non-digits)
- [0-9] — any digit
- \s — whitespace character
- ^ — start of string | $ — end of string

■ 30:00 – 35:00 | Hands-On Exercise — Clean a Raw Customer Table

Scenario:

You've received a CSV export of customer data. Mixed casing, extra whitespace, inconsistent phone formats. Your job: produce a clean SELECT in one query.

Raw Data (paste into notebook):

```
SELECT *
FROM (VALUES
  ('  alice smith  ', 'ALICE@GMAIL.COM',   '(555) 123-4567'),
  ('BOB JONES',       'Bob@Yahoo.com  ',   '555.987.6543  '),
  (' carol WHITE',    ' carol@email.com',  '555 246 8101  ')
) AS raw_customers(full_name, email, phone);
```

Required Output Columns:

Column	Transformation
clean_name	TRIM + INITCAP
clean_email	TRIM + LOWER
clean_phone	REGEXP_REPLACE — digits only

Expected Output:

clean_name	clean_email	clean_phone
Alice Smith	alice@gmail.com	5551234567
Bob Jones	bob@yahoo.com	5559876543
Carol White	carol@email.com	5552468101

Common Errors:

Error	Fix
Forgot to TRIM before LOWER — spaces survive	LOWER(TRIM(email))
REGEXP_REPLACE missing the g flag	Add 'g' as 4th argument
|| with NULL returns NULL	Use COALESCE() or CONCAT() for NULL safety

Solution:

```
SELECT
  INITCAP(TRIM(full_name))                          AS clean_name,
  LOWER(TRIM(email))                                AS clean_email,
  REGEXP_REPLACE(TRIM(phone), '[^0-9]', '', 'g')   AS clean_phone
```

```
FROM (VALUES
  ('  alice smith  ', 'ALICE@GMAIL.COM',   '(555) 123-4567'),
  ('BOB JONES',       'Bob@Yahoo.com  ',   '555.987.6543  '),
  (' carol WHITE',    ' carol@email.com',  '555 246 8101  ')
) AS raw_customers(full_name, email, phone);
```

Recap

✓ LOWER() / UPPER() / INITCAP() — normalize string casing

✓ TRIM() strips invisible whitespace that breaks comparisons

✓ CONCAT() handles NULLs gracefully; || does not

✓ LENGTH() validates field lengths; SUBSTRING() slices strings

✓ LIKE is case-sensitive; ILIKE (PostgreSQL) is case-insensitive

✓ REPLACE() swaps fixed strings; REGEXP_REPLACE() uses regex patterns

✓ String cleaning is the core of the staging transform layer

Day 7 Preview — Date & Time Basics

- DATE and TIMESTAMP types in PostgreSQL
- NOW(), CURRENT_DATE, CURRENT_TIMESTAMP
- Date arithmetic with INTERVAL — 'give me the last 30 days'
- Extracting year, month, day from a timestamp
- Pipeline use case: incremental loads using date filters

Camera Script — Sign Off:

"String functions are the backbone of the transform layer. Every pipeline you write will use at least five of these. Run the practice problems — especially Problem 9, which is a real staging transform. Tomorrow we move to dates and times, which is where pipelines really get interesting. See you on Day 7."

Appendix — Quick Reference Card

Function	Syntax	What It Does
LOWER	LOWER(str)	Convert to all lowercase
UPPER	UPPER(str)	Convert to all uppercase
INITCAP	INITCAP(str)	Title case — capitalize each word
TRIM	TRIM(str)	Remove leading + trailing whitespace
LTRIM / RTRIM	LTRIM(str) / RTRIM(str)	Left or right whitespace only
LENGTH	LENGTH(str)	Count characters in a string
SUBSTRING	SUBSTRING(str FROM n FOR len)	Slice a portion of a string (1-indexed)
CONCAT	CONCAT(a, b, ...)	Join strings; NULLs skipped silently
||	a || b	Join strings; NULL infects result
LIKE	col LIKE pattern	Case-sensitive pattern match (%, _)
ILIKE	col ILIKE pattern	Case-insensitive pattern match (PostgreSQL only)
REPLACE	REPLACE(str, old, new)	Swap a fixed substring
REGEXP_REPLACE	REGEXP_REPLACE(str, pat, rep, 'g')	Regex-powered replacement